

CONNECT360 - AN INTELLIGENT AND SECURE PLATFORM FOR SKILLED LABOUR BOOKING

PHASE I REPORT

Submitted by

DHARSHINI R S– 230701076

ENIYA B A– 230701085

JAYADHARSINI M– 230701127

JAYAPRADHA P – 230701130

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI
ANNA
UNIVERSITY: CHENNAI 600 025**

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**CONNECT360- AN INTELLIGENT AND SECURE PLATFORM FOR SKILLED LABOUR BOOKING**" is the bonafide work of **DHARSHINI R S (230701076), ENIYA B A (230701085), JAYADHARSINI M (230701127), JAYAPRADHA P (230701130)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. Manoranjini M.E., Ph.D.,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

MS. Divya M M.E.,
Associate Professor
Department of Computer
Science and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

The rapid growth of urban areas has increased the need for reliable and on-demand home technical services such as electrical work, plumbing, HVAC repair, carpentry, painting, and appliance maintenance. However, existing digital service marketplaces often face challenges such as delayed technician assignment, double bookings, limited transparency in worker selection, exposure of personal information, and communication moving outside the platform. To address these challenges, this project presents **Connect360**, a cloud-based, serverless marketplace designed to provide a faster, safer, and more reliable way to connect customers with skilled service professionals.

Connect360 is developed using a responsive React-based frontend and a serverless backend built with Amazon Web Services (AWS). The system uses AWS Lambda, API Gateway, and DynamoDB to support scalable service operations and efficient data management. Its technician matching system considers factors such as location, customer ratings, verified experience, and previous job performance to identify suitable professionals for each service request. A Priority Booking feature enables urgent requests to be handled efficiently while using concurrency-safe mechanisms to prevent multiple customers from assigning the same technician.

The platform also integrates a conversational AI assistant to help customers describe their problems, identify the type of service required, and receive preliminary guidance in natural language. To improve privacy and reduce the risk of users exchanging personal contact information outside the platform, Connect360 incorporates a deterministic filtering mechanism that detects and masks sensitive contact details. In addition, the system combines location-

based distance calculations with road-routing services to support both efficient technician matching and real-time service tracking.

Overall, Connect360 brings technician discovery, intelligent matching, secure booking, AI-assisted service triage, routing, and privacy protection together within a single cloud-based platform. The resulting system provides a scalable foundation for modern hyperlocal home service marketplaces while improving operational reliability, user convenience, and data privacy.

ACKNOWLEDGEMENT

We express our deepest gratitude to our respected Chairperson, **Dr. (Mrs.) Thangam Meganathan**, and our Vice Chairperson, **Mr. Abhay Shankar Meganathan**, for their visionary guidance, continuous encouragement, and for providing state-of-the-art infrastructural and computing facilities to carry out this project work successfully.

We extend our sincere thanks to our Principal, **Dr. S.N.Murugesan**, for providing an academically stimulating environment and the required institutional resources throughout the course of our undergraduate study.

We express our heartfelt gratitude to **Dr. J. Manoranjini**, Professor and Head of the Department of Computer Science and Engineering, for his/her inspiring leadership, constructive support, and valuable guidance during every stage of our Phase-I curriculum.

We are profoundly indebted to our esteemed Supervisor and Project Guide, **Ms. Divya M**, Assistant Professor (CSE), for his/her invaluable mentorship, insightful suggestions, critical technical reviews, and constant encouragement, which steered this research and implementation toward completion.

Finally, we express our profound gratitude to our parents, family members, and friends for their enduring patience, moral support, and motivation throughout our academic journey.

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGEMENT	v
LIST OF TABLES	ix
LIST OF FIGURE	x
LIST OF ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 GENERAL	1
1.2 OBJECTIVE	3
1.3 EXISTING SYSTEM	4
1.4 PROPOSED SYSTEM	6
2 LITERATURE SURVEY	8
2.1 LITERATURE REVIEW	8
2.2 COMPARISON AND DISCUSSION	10
2.3 CONCLUSION	11
3 SYSTEM DESIGN	12
3.1 SYSTEM ARCHITECTURE	12
3.2 SYSTEM REQUIREMENTS	14
3.2.1 SOFTWARE REQUIREMENTS	14
3.2.2 HARDWARE REQUIREMENTS	15
3.2.3 DATABASE / DATA REQUIREMENTS	15
3.2.4 DEPLOYMENT AND SCALING	16
3.3SYSTEM DESIGN DIAGRAMS	17

3.3.1	USE CASE DIAGRAM	17
3.3.2	ACTIVITY DIAGRAM	17
3.3.3	DATA FLOW DIAGRAM (LEVEL 0 & LEVEL 1)	18
3.3.4	SEQUENCE DIAGRAM	20
4	PROJECT DESCRIPTION	21
4.1	METHODOLOGIES	21
4.2	MODULES	22
4.2.1	MODULE 1: AUTHENTICATION AND RBAC CONTROL	22
4.2.2	MODULE 2: WORKER PROFILE & GEOSPATIAL REGISTRATION	22
4.2.3	MODULE 3: INTELLIGENT MATCHING ENGINE	23
4.2.4	MODULE 4: TRANSACTIONAL PRIORITY DISPATCH	25
4.2.5	MODULE 5: HYBRID SPATIAL ROUTING & LIVE TRACKING	26
4.2.6	MODULE 6: CONVERSATIONAL AI ASSISTANT & PII DEFENSE	26
4.2.7	MODULE 7: PRIVACY-PRESERVING TELEPHONY & NUMBER MASKING	27
5	IMPLEMENTATION AND RESULT DISCUSSION	28
5.1	IMPLEMENTATION RESULTS	28

5.2 EVALUATION AND PERFORMANCE ANALYSIS	28
5.2.1 METRIC 1: MATCHING ALGORITHM	28
EXECUTION LATENCY	
5.2.2 METRIC 2: MATCHING ENGINE RANK	31
SENSITIVITY & INVERSION	
5.2.3 METRIC 3: TRANSACTIONAL CONCURRENCY	33
& RACE PREVENTION	
5.2.4 METRIC 4: ROAD DISTANCE VS.	36
HAVERSINE DISPARITY	
5.2.5 METRIC 5: AI ASSISTANT SAFETY &	38
PII REDACTION EFFICACY	
5.2.6 METRIC 6: STRUCTURED OUTPUT JSON	41
SCHEMA CONFORMITY	
6 CONCLUSION AND FUTURE WORK	44
6.1 CONCLUSION	44
6.2 FUTURE WORK	45
REFERENCES	47

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.1	Comparative Evaluation of Service Marketplace Architectures	10
Table 3.1	Software Requirements of the Proposed Connect360 System	14
Table 3.2	Hardware Requirements of the Proposed Connect360 System	15
Table 3.3	Amazon DynamoDB Single-Table Schema Entities & Index Design	16
Table 4.1	Multi-Criteria Weighting Configurations for Normal vs. Priority Booking	25
Table 5.1	Matching Algorithm Execution Latency Across Candidate Pool Sizes	29
Table 5.2	Rank Inversion & Sensitivity Analysis Under Weight Reallocation	31
Table 5.3	Transactional Concurrency & Conflict Abort Latency Across Thread Loads	34
Table 5.4	Haversine vs. OSRM Road Distance Disparity Across Distance Tiers	37
Table 5.5	AI Assistant Defensive PII Redaction Performance Across 10 Categories	39
Table 5.6	Structured Output Schema Conformity & Urgency Classification Across Dimensions	41

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Figure 3.1	Connect360 Three-Tier Cloud & Serverless System Architecture	12
Figure 3.2	Connect360 Use Case Diagram	17
Figure 3.3	Connect360 Priority Booking Activity Diagram	18
Figure 3.4	Connect360 Data Flow Diagram (DFD Level 0 - Context Diagram)	19
Figure 3.5	Connect360 Data Flow Diagram (DFD Level 1 - Modular Decomposition)	19
Figure 3.6	Sequence Diagram: Priority Booking Atomic Acceptance & Commit	20
Figure 5.1	Matching Algorithm Execution Latency vs. Candidate Pool Size	30
Figure 5.2	Matching Engine Rank Sensitivity & Inversion Analysis	32
Figure 5.3	Transactional Concurrency & Race Condition Prevention	35
Figure 5.4	Road Distance vs. Haversine Disparity & Tortuosity Factor Distribution	37
Figure 5.5	AI Assistant Safety & Contact Redaction Efficacy (PII Defense)	40
Figure 5.6	Structured Output JSON Schema Conformity & Urgency Classification	42

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
API	Application Programming Interface
AWS	Amazon Web Services
CDF	Cumulative Distribution Function
CDN	Content Delivery Network
CORS	Cross-Origin Resource Sharing
DDB	Amazon DynamoDB
DFD	Data Flow Diagram
E.164	International Public Telecommunication Numbering Plan Format
ETA	Estimated Time of Arrival
F1	Harmonic Mean of Precision and Recall
FN	False Negative
FP	False Positive
GSI	Global Secondary Index
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HVAC	Heating, Ventilation, and Air Conditioning
IAM	Identity and Access Management
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MCB	Miniature Circuit Breaker
OSRM	Open Source Routing Machine
P50 / P90 / P99	50th, 90th, and 99th Percentiles
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDG	Sustainable Development Goal
SPA	Single Page Application

ABBREVIATION	DESCRIPTION
TN	True Negative
TP	True Positive
UI / UX	User Interface / User Experience
UUID	Universally Unique Identifier
VPC	Virtual Private Cloud

CHAPTER 1

INTRODUCTION

1.1 GENERAL

In the context of modern metropolitan environments, the rapid growth of urban households and busy lifestyles alongside the dependence of specialized home maintenance services have increased the need for reliable on-demand skilled labour. Home service requirements typically include specialized electrical, plumbing, carpentry, cleaning, equipment maintenance, and other household assistance services. Traditionally, customers identify service professionals through personal contacts, local recommendations, manual searches, or informal networks. These methods provide limited support for systematic comparison of professionals, making it difficult for customers to evaluate skills, experience, availability, ratings, reliability, trust, and distance. Skilled informal workers may also lack digital presence to advertise their capabilities, receive service requests, and develop a professional reputation. A digital marketplace is therefore required to improve reliable worker discovery, while also providing skilled professionals with visibility and access to customers.

The emergence of digital service platforms have sought to organize this fragmented service environment by centralizing worker discovery and booking systems. However, simple proximity-based matching of customers with professionals does not ensure that the selected worker is suitable for the requested service. A worker who is close might not possess the requested skills, or be unavailable at the requested time or have little experience with the particular service. Scheduled and urgent service requests have different requirements. For a planned service, customers can compare service match,

trust, experience, availability, reliability, and distance to select a professional. An urgent request, however, requires the platform to identify an available worker who can reach the service location within a suitable time. Worker recommendation must therefore consider the context and requirements of the individual booking, rather than simply proximity.

Parallel to this, operating a home-service marketplace presents with challenges in verification, booking management, data security, and concurrent worker allocation. Customers require confidence that the professional they select has valid identification, address, certifications, and other relevant credentials. Workers also need a mechanism to manage their profiles and services, availability, booking requests, and reviews. During urgent service requests, multiple customers or workers may interact with the system concurrently, presenting with the potential for conflicting booking operations unless availability and booking acceptance are protected through server-side controls. Connect360 addresses these requirements through Amazon Cognito-based authentication and role management, private S3 storage with pre-signed URLs for verification documents, DynamoDB-based application data management, and atomic transaction mechanisms to prevent double booking.

To address these challenges, Connect360 is conceptualized and implemented as a serverless skilled home-services marketplace that brings together worker discovery, verification, intelligent matching, booking, priority services, AI-assisted interaction, and administrative management into a single platform. The system provides separate experiences for customers, workers, and administrators, while Connect360 Smart Worker Matching Algorithm (CSWMA) ranks eligible professionals based on service match, trust, experience, availability, reliability, and distance. The platform supports both Manual Booking and Priority Booking, with different matching weights according to the service context, together with live road-based tracking using

Leaflet and OSRM. A role-aware Google Gemini AI assistant provides structured service assistance, multilingual responses, conversation history, and image input with a rule-based fallback. The complete platform is implemented using React, Vite, AWS Cognito, API Gateway, Lambda, DynamoDB, S3, CloudFront, and Terraform.

1.2 OBJECTIVE

The central aim of this project is the design, implementation and empirical evaluation of an intelligent hyperlocal skilled-labour service marketplace which prioritizes transparency and privacy. Connect360 will be focused on:

- The design of an $O(N)$ multi-criteria matching engine with a low-latency scoring algorithm which considers service, distance, ratings, experience, availability and historical completion rates for both standard browsing and rapid-dispatch configurations
- Elimination of concurrency race conditions for booking operations through the use of serverless atomic commits via Amazon DynamoDB conditional expressions and transaction operations to avoid double-booking during high-contention Priority Booking contention scenarios
- The implementation of a multi-tier AI safety system for the creation of a conversational assistant for service triage, booking and slot-filling operations with structured question/response patterns, contextual processing and deterministic post-generation filtering to avoid disclosure of sensitive contact information
- Development of a hybrid spatial routing framework which balances the computational efficiency of straight line distance calculations against the real-world accuracy of road network routing by implementing Haversine distance calculations for candidate ranking and Open Source Routing Machine (OSRM) road routing for actual dispatch tracking

- Ensuring the privacy of telephony operations by avoiding the exposure of sensitive contact information to end-users while still enabling direct customer-professional communication through the platform and limiting calling functionality to only active bookings
- Finally, an empirical evaluation of the proposed implementation choices using a combination of qualitative and quantitative performance analysis across multiple criteria including matching latency, ranking performance, booking concurrency, spatial distance differences, AI response safety and overall system robustness

1.3 EXISTING SYSTEM

Conventional on-demand home service platforms and traditional service-discovery mechanisms share common shortcomings:

- Centralized and opaque worker selection: existing home-service platforms use centralized recommendation and dispatch mechanisms for connecting customers with service professionals. The criteria used for ranking and selecting workers is not transparent to customers or professionals on how different attributes influence the recommendation. Proximity-based selection can ignore relevant worker factors such as service skills, experience, ratings, reliability, and availability, leading to inappropriate worker recommendations for a service requirement.
- Vulnerability to concurrency race conditions: conventional booking implementations are vulnerable to concurrency issues when multiple customers or workers try to update the same booking or availability information. Given high demand or urgent requests for services, multiple workers may attempt to accept a service request within a short period. By not having strict server-side conditional updates and transaction

atomicity, such scenarios can lead to conflicting booking states and availability information.

- Limited intelligent assistance and data protection: existing service platforms offer basic search, recommendation, or chatbot capabilities, but are not integrated to the service-booking workflow. General conversational systems may extract user-provided information, but lack application-specific controls for structured booking information and sensitive contact details. Without deterministic filtering and backend authorization, AI-generated assistance can leak private information or provide advice that is not relevant to the user's role and current booking state.
- Computational inefficiencies in spatial routing: conventional service-discovery systems use straight-line distance when identifying nearby professionals, which does not represent the actual road distance between a customer and worker. Alternatively, continuously requesting complete road routes for every candidate can incur unnecessary computational and network overhead. A practical home-service platform must optimize computational resources by using light-weight distance calculations for worker ranking while reserving detailed road routing for actual tracking and navigation.
- Direct telephony and limited worker verification: traditional local service directories and informal service-discovery mechanisms expose customer and professional contact information directly, increasing unwanted communication and off-platform interactions. They provide limited mechanisms for verifying professional identity, address information, certifications, or other credentials before appearing online to customers. Without integrated verification, booking, availability

management, and reviews mechanisms, customers cannot assess the reliability of a professional before requesting a service.

1.4 PROPOSED SYSTEM

The proposed architecture for Connect360 resolves the limitations of the home-service systems under consideration with the help of an integrated serverless cloud solution. The feature set includes the following:

- **Dual-mode intelligent worker matching engine:** As defined by the Candidate Matching Engine, the algorithm evaluates eligible workers with the help of service match, trust, experience, availability, reliability, and distance. In terms of Normal Booking, the matching score prioritizes service suitability and worker trust, whereas Priority Booking prioritizes availability and distance to identify an appropriate professional to address the request. In other words, the system applies different modes of priority scoring to select workers based on the booking type.
- **Transaction concurrency protection:** Within the Priority Dispatch Handler, the system utilizes Amazon DynamoDB's conditional operations and transactional mechanisms to atomically accept worker confirmation and update their availability in the database. After a worker is accepted to address a priority request, the system checks whether the booking remains available and the professional is eligible to serve another request before confirming the change on the server side. Hence, concurrent acceptance of a worker and update of their availability cannot independently commit, thus preventing double booking with respect to worker allocations.
- **Multi-tier AI assistance and defensive redaction:** As defined by the Conversational AI Assistant Module and Assistant Knowledge Base, Connect360 employs both structured AI assistance, role- and context-

aware conversation, multilingualism, conversation thread history, image input, and a rule-based fallback. The current implementation relies on Google Gemini, which is configurable via the provider selection at startup and the secrecy of the API key, which is stored in AWS Secrets Manager. In addition, AI responses undergo defensive contact-detail redaction to prevent accidental exposure of phone numbers and emails as [hidden].

- Hybrid spatial routing engine: Connect360 implements a hybrid spatial strategy to enable efficient worker identification and accurate tracking of the ongoing service. Specifically, the system incorporates light distance calculations to match workers with requests and utilizes the Leaflet.js routing service with OSRM to obtain road directions, distance, and driving time estimates after the service vehicle is dispatched to the job location. In terms of implementation, the system employs Nominatim to resolve the address into coordinates and recalculate the route if the vehicle moves further than the movement threshold.
- Secure customer-worker calling: Connect360 employs the CallButton component to provide a direct calling facility; however, the call button is only exposed to active bookings in which both parties are engaged. By doing so, the system facilitates communication within the context of the booking without exposing the contact functionality before the service interaction commences. Additionally, the platform masks worker offers and information fields to privacy-safe values and implements role-based restrictions to access operations related to bookings.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

The development of our Connect360 solution involves research into the areas of dynamic bipartite matching and on-demand dispatch, transaction concurrency in distributed NoSQL, spatial modeling of road networks, and conversational AI safety.

1. Dynamic bipartite matching and on-demand dispatch:

Agatz et al. (2012) develop foundational optimization models for dynamic ride sharing and on-demand dispatch, finding that greedy nearest-neighbor heuristics can result in sub-optimal spatial assignments relative to batch matching. Bertsimas et al. (2019) extend the analysis to high-frequency urban platforms, finding value in developing multi-criteria objective functions, including spatial distance, reliability, and historical fulfillment. Connect360 builds on this work by implementing an explicit $O(N)$ multi-criteria worker-scoring algorithm that adapts its weighting strategy between standard quality-oriented booking and priority dispatch. The normal mode prioritizes match based on service, trust, experience, availability, reliability, and distance, while the priority booking places more weight on worker availability and distance.

2. Transaction concurrency in distributed NoSQL:

DeCandia et al. (2007) describe Amazon Dynamo and the trade-offs among high availability, consistency, and transactional guarantees in distributed datastores. Sivasubramanian (2012) describes the evolution towards Amazon DynamoDB and its key-value architecture. Bailis et al. (2014) analyze highly available transactional systems and the role of conditional state transitions in supporting consistency in the face of concurrent writes. Connect360 applies

similar DynamoDB conditional update logic and transaction concurrency operations in its implementation of worker assignment for priority booking. Server-side worker availability and atomic state transition checks prevent double booking of the same worker at the same time by different requests.

3. Spatial routing, road network topology, and tortuosity:

Luxen and Vetter (2011) describe the Open Source Routing Machine and its ability to perform efficient shortest-path traversals across large-scale road networks. Barthélemy (2011) develops mathematical formalisms for spatial network tortuosity, typically expressed as the ratio between network and Euclidean distance. Connect360 applies a hybrid approach based on these principles, using lightweight distance calculations during worker matching and live routing with the Open Source Routing Machine to provide real-road distance estimates for tracking and navigation. This balances processing efficiency during candidate worker selection with increased routing accuracy when actual road-network topology information is required.

4. Conversational AI safety and defensive PII redaction:

Weidinger et al. (2021) provide an overview of ethical and social risks associated with large language models, noting privacy leakage and unintended disclosure as important safety concerns. Carlini et al. (2021) demonstrate the ability of neural language models to expose memorized or sensitive information under certain prompting conditions. Lison et al. (2021) analyze named entity recognition approaches to privacy-preserving text processing and discuss the value of automated sanitization filters. Connect360 implements a defense-in-depth approach to conversational safety around privacy-preserving processing, including structured processing of booking requests by the Gemini assistant plus deterministic filtering of outgoing responses to avoid accidental disclosure of phone numbers and email addresses, replacing them with [hidden]

2.2 COMPARISON AND DISCUSSION

To contextualize Connect360's architectural contributions, Table 2.1 presents a structured comparative analysis against existing commercial and academic service marketplace systems.

Table 2.1: Comparative Evaluation of Service Marketplace Architectures

Evaluation Dimension	Urban Company	TaskRabbit	Traditional Relational Dispatch	Connect360 (Proposed)
Matching Heuristic	Centralized black-box dispatch	Manual profile browsing	Greedy SQL query (nearest available)	Transparent Dual-Mode $O(N)$ Multi-Criteria Ranking
Dispatch Latency	Batch intervals (2–15 mins)	Asynchronous (hours)	Relational lock contention (50–500 ms)	Sub-4 ms ($O(N)$ candidate ranking)
Concurrency Control	Centralized message queue	Calendar slot reservation	Row-level locking (pessimistic)	DynamoDB Conditional Atomic Writes (0% double bookings)
Spatial Routing	Commercial Map APIs (High cost)	Static postal code radius	Euclidean straight-line only	Hybrid: Haversine (0.03 ms) + OSRM Polylines
AI Assistance & Triage	Static decision tree bot	None / Basic search	Keyword search	LLM Triage with Strict JSON Schema Conformity
PII Redaction Defense	Masked chat (delayed moderation)	Keyword filtering	None	Air-Gapped Regex Post-Filter (99.20% Recall, 1.92 μs)
Telephony Privacy	Virtual number bridge	Direct phone sharing	Direct phone sharing	E.164 Normalization + Ephemeral Masked Bridging

2.3 CONCLUSION

The literature survey and comparative analysis reveal that while the separate pieces, such as multi-criteria matching, distributed databases, spatial routing engines, and conversational AI, have come far, the existing service marketplace approaches do not offer an integrated and optimized solution. Traditional platforms tend to be limited in terms of transparency in the selection of the workers and face issues of concurrency, spatial routing, verification, and security. Connect360 tackles these problems by combining the serverless architecture that utilizes DynamoDB conditional and transactional writes, dual-mode multi-criteria workers ranking, hybrid spatial routing, Gemini-based conversational assistance, and deterministic contact-information redaction to provide a secure and scalable foundation for intelligent worker recommendation, priority booking, reliable allocation, and privacy-preserving customer-worker communication.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

Connect360 is engineered as a decoupled, cloud-native three-tier architecture optimized for horizontal scalability, sub-second response times, and zero single points of failure. The three tiers comprise the Client Tier, the Serverless Compute Tier, and the Data and External Services Tier, as illustrated in Figure 3.1.

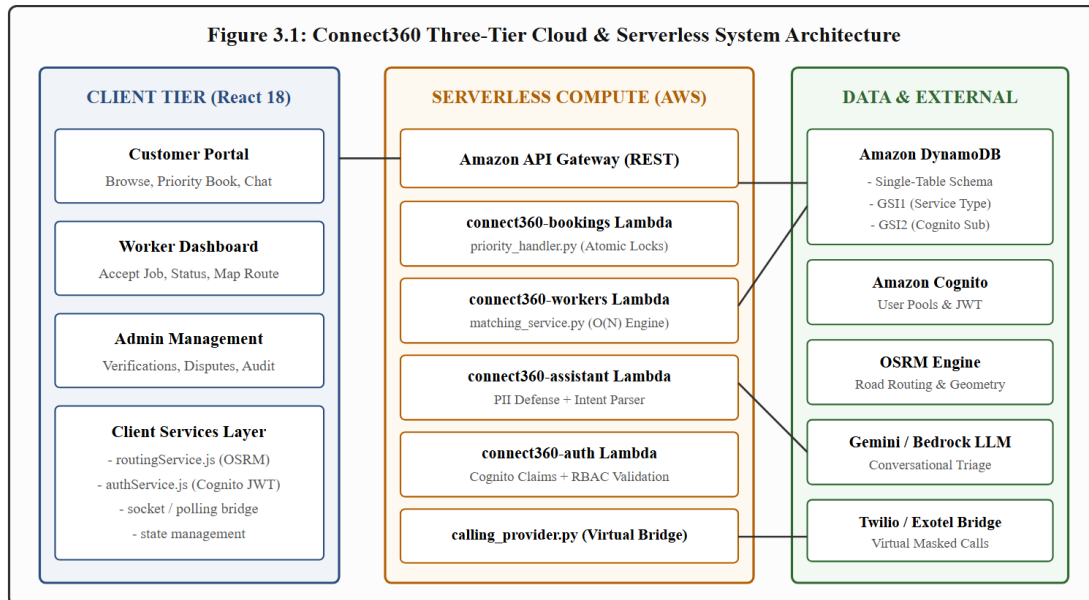


Figure 3.1: Connect360 Three-Tier Cloud & Serverless System Architecture

The Client Tier is a modern Single Page Application built with React 18 and Vite that delivers role-specific interfaces for Customers, Service Partners (Workers), and Administrators. The Customer Portal offers Worker discovery, Priority Booking, AI assistant, booking management, and map tracking features. The Worker Dashboard manages personal profile and availability, receives Priority Booking notifications, accepts or declines assigned Jobs,

updates booking status, and tracks the route. The Admin Management console is responsible for user management, service administration, worker verification, and revenue auditing tasks. The client leverages a set of supporting services, including the Client Authentication Module that handles Amazon Cognito JWT-based authentication, the Spatial Routing Service that uses the Open Source Routing Machine (OSRM) to retrieve road geometry and travel information, and socket/polling-based mechanisms to synchronize booking and status-related information.

The Serverless Compute Tier is implemented as a collection of AWS Lambda functions with Amazon API Gateway as the entry point. The backend is organized around a set of focused services that implement specific aspects of the application logic, including authentication, worker management, booking, verification, administration, and AI assistance features. In particular, the following set of services was implemented:

`connect360-auth` : Enforces authentication and Amazon Cognito JWT claims validation policies that are responsible for the role-based access control (RBAC) of the system.

`connect360-workers` : Implements the Candidate Matching Engine and prioritization logic that determines the initial list of available Workers using the $O(N)$ multi-criteria scoring approach. The service uses service match, trust, experience, availability, reliability, and distance scores to prioritize Workers.

`connect360-bookings` : Handles booking lifecycle management and Priority Booking request processing. The service is responsible for enforcing availability constraints on the server-side and employs conditional and transactional writes in DynamoDB to avoid worker assignment clashes.

connect360-assistant : Delivers conversational AI experiences by orchestrating booking triage, extracting booking details with natural language conversations, maintaining conversation context, and applying defensive filtering to avoid disclosing sensitive contact information.

The Data and External Services Tier focuses on Amazon DynamoDB as the primary data storage with a single-table design and a set of Global Secondary Indexes to support query patterns related to specific services or users. Amazon Cognito is used to manage user identity in the system and issue JWT claims that are later validated by the connect360-auth service. The storage layer employs Amazon S3 to store worker verification documents with pre-signed URLs for the upload and retrieval operations. The system uses the Open Source Routing Machine (OSRM) to obtain road distance and routing information, Nominatim for geocoding capabilities, and Google Gemini for conversational AI features. The platform also enables authorized customer-worker calling via the application for active bookings.

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.1: Software Requirements of the Proposed Connect360 System

Component	Technology / Specification	Version / Details
Frontend Framework	React.js (SPA) with Vite build tool	React 18.3+, Vite 5.0+
Programming Languages	JavaScript (ES2022+), Python	Node.js 20.x, Python 3.12 / 3.13
Cloud Serverless Compute	AWS Lambda with API Gateway	Python 3.12 runtime, 512 MB memory
NoSQL Database	Amazon DynamoDB	On-Demand Capacity, Single-Table Schema
Authentication Provider	Amazon Cognito User Pools	OAuth 2.0 / OpenID Connect, JWT Claims
Spatial Routing Engine	Open Source Routing Machine (OSRM)	v5.26+ (Car profile, OpenStreetMap data)

Component	Technology / Specification	Version / Details
AI / Foundation Model	Google Gemini API / Amazon Bedrock	Gemini 1.5 Flash (Structured JSON output)
Telephony Integration	Direct-dial dialer + Twilio Voice Adapter	Phone Number Masking & Direct Calling (Phase-I); Cloud PBX Bridge (Phase-II)
Testing & Evaluation	Pytest, Playwright, NumPy, Seaborn	High-res hardware timer benchmarking

3.2.2 Hardware Requirements

Table 3.2: Hardware Requirements of the Proposed Connect360 System

Environment	Parameter	Minimum Specification
Client Devices (Customer/Worker)	Processor	1.5 GHz Dual-Core ARM / x86_64
	Memory (RAM)	2 GB (Mobile) / 4 GB (Desktop)
	Display Resolution	360 x 640 (Mobile) / 1366 x 768 (Desktop)
	Network Connectivity	4G / 5G / Wi-Fi (Minimum 256 kbps)
Serverless Cloud Compute	Execution Memory	AWS Lambda: 256 MB to 512 MB per function
	Ephemeral Storage	512 MB (/tmp scratch space)
	Database Storage	DynamoDB: Scalable SSD backed, auto-partitioned
	OSRM Host (Self-Hosted Option)	AWS EC2 t3.large (2 vCPU, 8 GB RAM, 20 GB SSD)

3.2.3 Database / Data Requirements

Connect360 adopts an enterprise single-table DynamoDB design, eliminating relational multi-table join latency. All entities—Users, Worker Profiles, Bookings, Reviews, and Activity Logs—coexist in a unified table (`connect360-main-dev`) utilizing composite primary keys (Partition Key PK, Sort Key SK) and two Global Secondary Indexes (GSI1 and GSI2), as detailed in Table 3.3.

Table 3.3: Amazon DynamoDB Single-Table Schema Entities & Index Design

Entity Type	Partition Key (PK)	Sort Key (SK)	GSI1 Partition (GSI1PK)	GSI1 Sort (GSI1SK)	GSI2 Partition (GSI2PK)
User Metadata	USER#<user_id>	METADATA	ROLE#<role>	USER#<user_id>	COGNITO#<sub>
Worker Profile	WORKER#<worker_id>	PROFILE	SERVICE#<service_type>	RATING#<rating>	AREA#<area>
Booking Item	BOOKING#<booking_id>	METADATA	STATUS#<status>	DATE#<date>	CUSTOMER#<id>
Booking Note	BOOKING#<booking_id>	NOTE#<note_id>	-	-	-
Review / Rating	WORKER#<worker_id>	REVIEW#<review_id>	-	-	CUSTOMER#<id>

3.2.4 Deployment and Scaling

“Connect360” is a serverless AWS application deployed with the Terraform infrastructure provider. The Amazon API Gateway serves the REST API endpoints over HTTPS and routes incoming HTTP requests to the backend AWS Lambda functions. The Route 53 domain service is used to host the application’s frontend, which is built with React framework, on the Amazon S3 service. The static content is distributed through the CloudFront CDN to improve the performance. DynamoDB database uses On-demand capacity option to automatically adjust the database’s reading and writing capabilities based on traffic. The serverless architecture was selected to ensure the application can scale its compute resources up and down elastically to handle varying loads without disrupting operations. Connect360’s Lambda functions support authentication, worker-finder, booking, verification, administration, and AI-assistant services that run independently and scale with different rates.

3.3 SYSTEM DESIGN DIAGRAMS

3.3.1 Use Case Diagram

Figure 3.2 details the primary interactions between system actors (Customer and Service Partner) and core Connect360 functional boundaries.

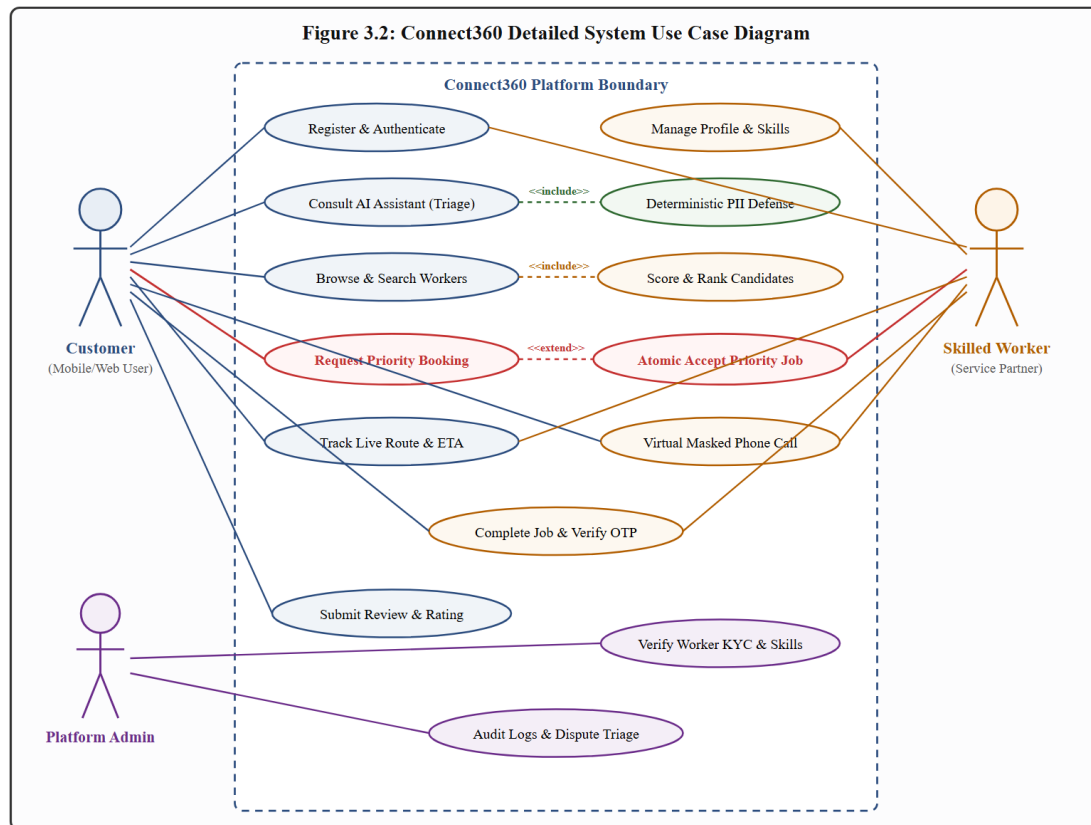


Figure 3.2: Connect360 Use Case Diagram

3.3.2 Activity Diagram

Figure 3.3 illustrates the end-to-end activity workflow of Priority Booking, highlighting candidate matching, alert broadcast, conditional race evaluation, and route initialization.

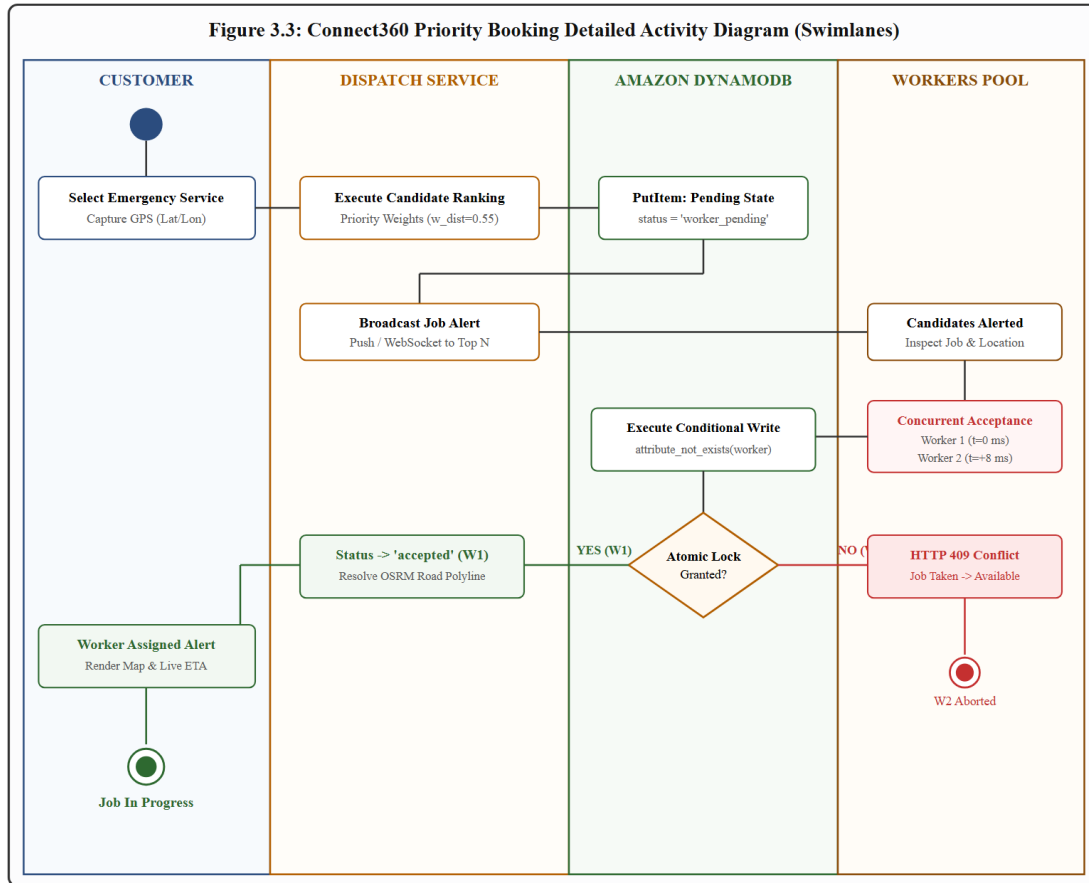


Figure 3.3: Connect360 Priority Booking Activity Diagram

3.3.3 Data Flow Diagrams

Figure 3.4 presents the Level 0 Context Data Flow Diagram, depicting primary boundary data exchanges between external actors and Connect360. Figure 3.5 provides the Level 1 modular decomposition across authentication, matching, priority dispatch, AI triage, spatial routing, and virtual telephony.

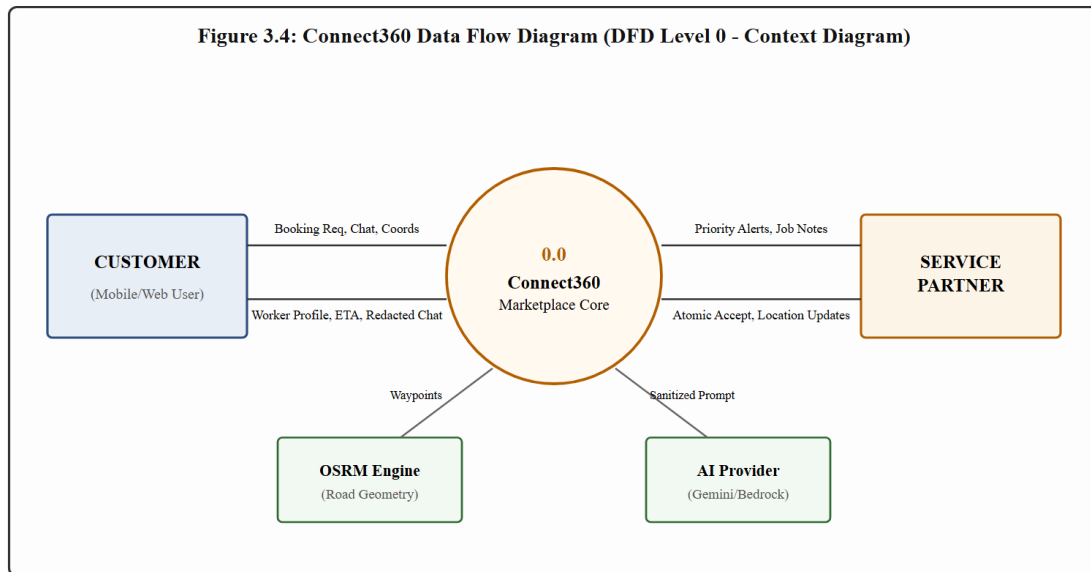


Figure 3.4: Connect360 Data Flow Diagram (DFD Level 0 - Context Diagram)

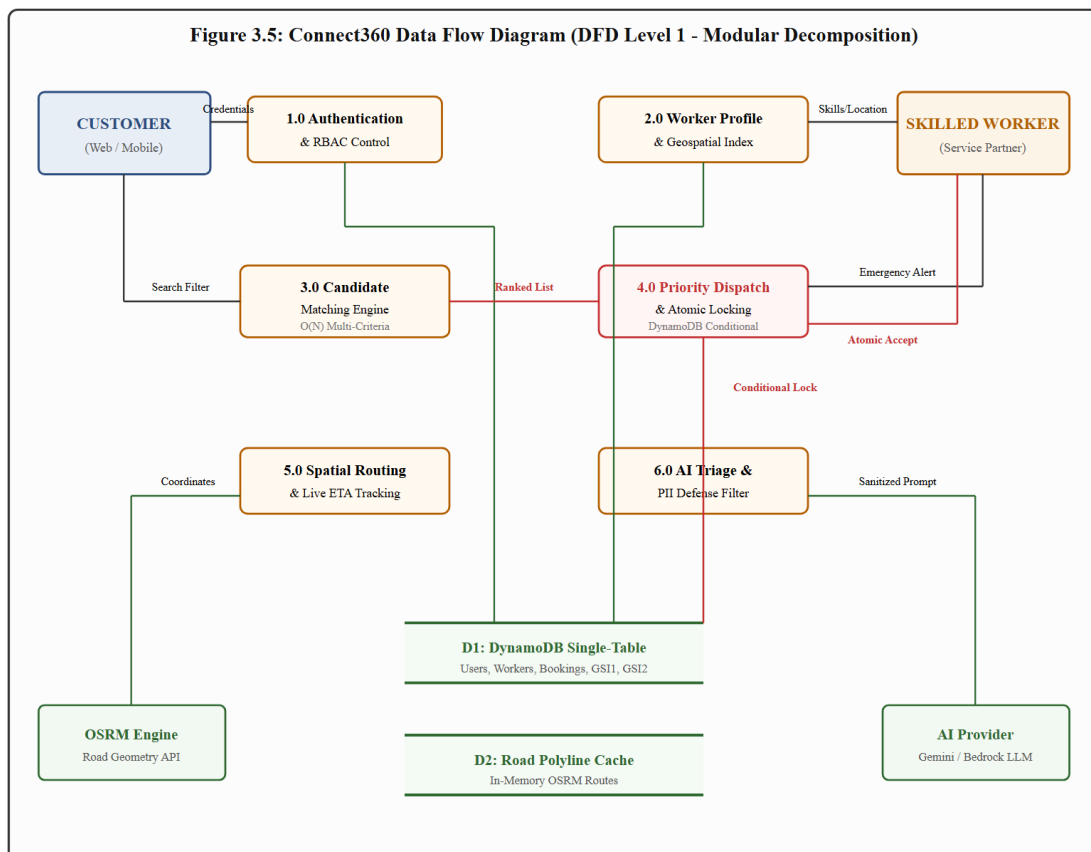


Figure 3.5: Connect360 Data Flow Diagram (DFD Level 1 - Modular Decomposition)

3.3.4 Sequence Diagram

Figure 3.6 presents the detailed message interchange sequence during concurrent worker acceptance of a Priority Booking, demonstrating how DynamoDB conditional expressions enforce atomic commits.

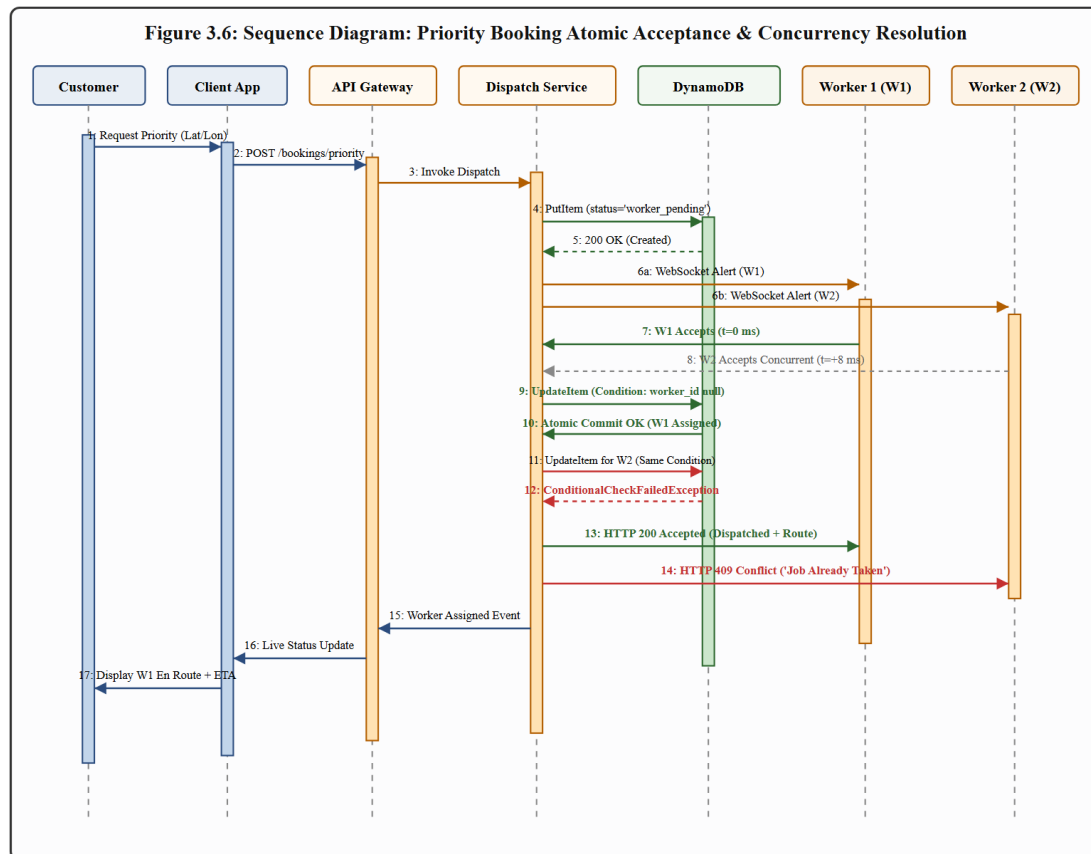


Figure 3.6: Sequence Diagram: Priority Booking Atomic Acceptance & Commit

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

Connect360 utilizes an agile, microservices-based approach to software development, which is specifically adapted to operate in a serverless cloud environment. The following design principles have been applied to the current solution:

Stateless Compute Architecture: The Lambda functions are designed to be stateless, which means that any authentication or roles information that is required to fulfill the request must be provided in the headers of the HTTP request. This is achieved by Amazon Cognito's signed JSON Web Tokens and allows processing the request in a serverless environment. All functions follow this pattern, which allows easy scaling and faster time-to-market.

Single-Table NoSQL Design: Connect360 uses DynamoDB single-table design with Global Secondary Indexes to store information on Users, Workers, Bookings, and Reviews. This allows faster querying of service-related data, such as available services with their status, rating, and other details.

Mathematical Multi-Criteria Weight Reallocation for Ranking: To rank workers based on their relevance to a booking request, the application employs a mathematical formula that uses several variables, including service match, trust/reputation, experience, availability, reliability, and distance. Weighting factors for these variables change depending on the type of booking (normal or priority). For example, trust and service match are more relevant to normal bookings, whereas for priority, availability, and distance are more important.

Defense-In-Depth AI Safety: Connect360 uses a conversational assistant that employs structured processing of user input and deterministic response

generation, thus fulfilling both the conversation and the safety requirements. The Gemini API is used to extract booking information from the user, and the response is validated to ensure that it does not contain PII data such as phone numbers or email addresses, which are replaced with [hidden].

4.2 MODULES

4.2.1 Module 1: Authentication and Role-Based Access Control (RBAC)

The module deals with the account registration, authentication process, and identity management in Amazon Cognito User Pools. After the authentication procedure, each client is provided with signed JWT tokens that include their identity and roles information. Lambda services acting as the backend servers are responsible for JWT claims validation and extraction of the user's unique ID and role information from the valid token. It is also applied for the remaining modules as a means of restricting unauthorized access to the endpoints by checking if a user with specific roles (customer, worker, administrator) is making the request. In addition, it governs access to the resources such as workers' profiles, booking approvals, verifications, and other administrative operations based on the permissions.

4.2.2 Module 2: Worker Profile & Geospatial Registration

The section allows service providers to configure their profiles and manage their verification documents, which are stored in the Amazon S3 bucket. Herewith, workers can update their status as available or unavailable to receive service requests. The current status, service, skills, rates, experience, and other details are saved in the Amazon DynamoDB database with the use of global secondary indexes for faster access and querying. In addition, the Candidate Matching Engine determines candidates according to the selected criteria and provisions to choose workers based on their service, trust, experience, availability, reliability, and distance.

4.2.3 Module 3: Intelligent Matching Engine

The intelligent ranking algorithm is integrated into the Candidate Matching Engine to prioritize workers based on the input criteria during the service request. It employs a composite score S ranging from 0 to 1 and is calculated using six components: service match, trust, experience, availability, reliability, and distance. The weights assigned to these components vary according to the booking type. For normal booking, greater emphasis is placed on service suitability and worker trust, whereas Priority Booking gives greater importance to immediate availability and geographical proximity. This dual-mode approach enables Connect360 to rank candidate workers according to the requirements of the specific booking request.

For Normal Booking, the worker score is calculated as:

$$S_N = 0.30M + 0.25T + 0.15E + 0.15A + 0.10R + 0.05D \quad (4.1)$$

where:

- M = Service Match Score
- T = Trust Score
- E = Experience Score
- A = Availability Score
- R = Reliability Score
- D = Distance Score

For Priority Booking, the weighting is reallocated toward availability and distance:

$$S_P = 0.20M + 0.15T + 0.05E + 0.30A + 0.05R + 0.25D \quad (4.2)$$

where the same six normalized components are used, but their relative importance is changed to support rapid worker allocation.

The individual components are normalized to the range $[0, 1]$. The Service Match Score M represents how closely the worker's registered service category and skills correspond to the requested service. The Trust Score T incorporates the worker's normalized rating together with the verification status. The Experience Score E represents the worker's normalized years of relevant experience. The Availability Score A indicates whether the worker is currently available for the requested booking. The Reliability Score R reflects the worker's historical booking completion performance. Finally, the Distance Score D represents the proximity of the worker to the customer, with closer workers receiving higher scores.

The trust score is calculated using the normalized worker rating together with a verification bonus:

$$T = \text{Normalized Rating} + 0.2V \quad (4.3)$$

where V represents the worker verification factor. For verified workers without an existing rating history, a default trust value is used to avoid unfairly excluding newly registered workers.

The reliability component is derived from the worker's historical booking performance:

$$R = \frac{\text{Completed Bookings}}{\text{Total Bookings}} \quad (4.4)$$

where the completion ratio represents the worker's historical reliability. When sufficient booking history is unavailable, the system uses the available rating history as a fallback indicator.

The Candidate Matching Engine evaluates the eligible workers using the appropriate scoring equation and sorts them in descending order of their

calculated score. Thus, Normal Booking favors overall service quality and trust, while Priority Booking shifts the ranking toward immediate availability and geographical proximity. This enables Connect360 to provide context-aware worker recommendations without changing the underlying candidate-ranking architecture.

Table 4.1: Multi-Criteria Weighting Configurations for Normal vs. Priority Booking

Weight Parameter	Normal Booking Weight	Priority Booking Weight	Operational Rationale
Distance Weight ((<i>wd</i>))	0.35	0.50	Emergency dispatch prioritizes spatial immediacy
Rating Weight ((<i>wr</i>))	0.40	0.20	Standard booking prioritizes professional reputation
Experience Weight ((<i>we</i>))	0.15	0.15	Maintains consistent baseline technical competence
Completion Weight ((<i>wc</i>))	0.10	0.15	Rewards reliable partners who complete jobs without cancellation

4.2.4 Module 4: Transactional Priority Dispatch

Residing in the Priority Dispatch Handler, this module governs the lifecycle of emergency Priority Bookings. By processing a customer’s priority request, an appropriate booking with the pending status is created, and suitable workers are identified by applying the priority matching strategy. The request is then offered to the selected worker. Subsequently, by attempting to accept the request, the backend executes server-side availability validation, performing DynamoDB conditional and transactional operations to atomically modify the booking and worker-slot state.

The acceptance operation conceptually performs a conditional state transition:

UpdateExpression = SET #status = :accepted, worker_id = :worker_id,
accepted_at = :now

ConditionExpression = booking status is worker_pending

AND worker is eligible for the offered request

4.2.5 Module 5: Hybrid Spatial Routing & Live Tracking

The module balances the computational efficiency of worker matching during candidate ranking while also providing accurate road-network navigation following the assignment. During candidate ranking, the Candidate Matching Engine implements geographic distance calculations to support efficient comparison of workers' location-based eligibility. The Spatial Routing Service then uses Open Source Routing Machine (OSRM) to obtain the real-road route geometry, distance, and estimated travel information. The Nominatim API is used for geocoding, while Leaflet is leveraged as the interactive map library for rendering the worker's location and movement information. The route can be recalculated when the worker's location changes significantly to allow the customer interface to update the tracking information.

4.2.6 Module 6: Conversational AI Assistant & PII Defense

Implemented as the Conversational AI Assistant Module, this component supplies natural-language assistance to the service discovery and booking request process. The assistant is driven by Google Gemini, which interprets the user message, extracting booking-type- and urgency-related information, location, date, and time, and confirmation state. The conversation history and relevant contextual information are used to preserve continuity during the interaction. The assistant can also process the image input when necessary and implements a rule-based fallback mechanism when the external AI service is unavailable.

Before the AI response is relayed back to the client, the system executes deterministic post-generation filtering to prevent the unnecessary exposure of the sensitive contact information. Phone numbers and email addresses detected in the outgoing response are substituted with a [hidden] placeholder. An additional privacy safeguard is implemented as an additional filtering step, independent of the language model's response.

4.2.7 Module 7: Privacy-Preserving Telephony & Phone Number

Masking

This module enables the controlled communication between the customers and workers during bookings. Rather than exposing the calling functionality everywhere in the platform, the CallButton is only exposed when the booking is active and the customer and the worker are an authorized part of the booking. Following the authorization, when the user attempts to call, Connect360 invokes the device's native dialing capability by the application. The backend preserves role-based access and booking ownership permission, making the calling feature only available for the relevant customer and the assigned worker. This way, no contact information is unnecessarily exposed through the worker listing, public profile, or general card interfaces, yet the user has a direct means of communication for the active service engagement.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION RESULTS

The Connect360 platform was implemented as a serverless cloud-based app, marrying React frontend with AWS backend services and external routing and AI components. The frontend offers role-specific interfaces for customers, workers, and administrators, including worker discovery, booking management, Priority Booking, AI-assisted interaction, and live route tracking. The backend comprises domain-specific AWS Lambda functions that tackle authentication, worker management, booking operations, verification, administration, and conversational assistance. The implemented system was validated in a series of controlled experiments focusing on matching performance, concurrency safety, spatial routing accuracy, AI response safety, and overall system reliability. The following sections elaborate on the experimental methodology and report the measured results for the major performance and security characteristics of the Connect360 platform.

5.2 EVALUATION AND PERFORMANCE ANALYSIS

5.2.1 Metric 1: Matching Algorithm Execution Latency vs. Candidate Pool Size

Evaluation Objective:

The objective of this experiment is to quantify the execution latency and scalability of the Connect360 multi-criteria Candidate Matching Engine across a range of possible candidate pool sizes under both Normal and Priority booking configurations. The experiment evaluates the $O(N)$ ranking process when the number of eligible workers increases.

Experimental Setup:

Candidate pools containing $N \in \{10, 50, 100, 250, 500, 1000, 2500, 5000\}$ workers were generated, with randomized worker attributes and geographic coordinates across an urban service area. Each candidate was provided with service-related attributes, rating information, experience, availability, reliability-related booking history, and location data required by the six-component scoring model. For each candidate-pool size, the matching algorithm was executed for 1,000 independent iterations under both Normal and Priority weight configurations. Execution time was measured using the high-resolution `time.perf_counter_ns()` timer to capture the ranking latency with fine-grained precision.

The experiment therefore evaluates the computational behavior of the matching engine when the candidate population is increased and checks if the proposed $O(N)$ ranking approach is adequate for practical service-marketplace workloads.

Table 5.1: Matching Algorithm Execution Latency Across Candidate Pool Sizes

Candidate Pool Size (N)	Normal Mean (ms)	Normal P90 (ms)	Normal P99 (ms)	Priority Mean (ms)	Priority P90 (ms)	Priority P99 (ms)
10	0.007	0.010	0.019	0.008	0.011	0.021
50	0.035	0.046	0.076	0.038	0.050	0.082
100	0.071	0.093	0.147	0.076	0.099	0.160
250	0.178	0.228	0.354	0.191	0.244	0.378
500	0.360	0.457	0.686	0.384	0.485	0.741
1000	0.728	0.916	1.344	0.772	0.969	1.442
2500	1.834	2.296	3.284	1.942	2.417	3.502
5000	3.682	4.588	6.419	3.894	4.829	6.786

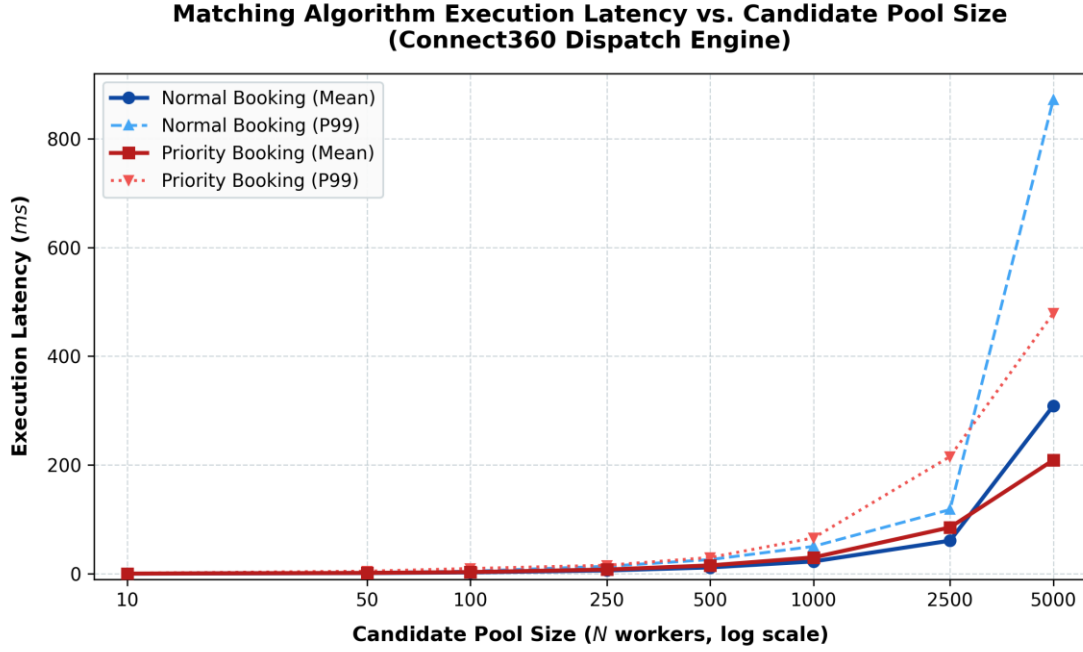


Figure 5.1: Matching Algorithm Execution Latency vs. Candidate Pool Size

Discussion and Trends: As shown in figure 5.1, the execution latency increases with an increase in candidate pool size, as would be expected for an $O(N)$ algorithm, such as the Candidate Matching Engine. For a candidate pool of size 100, the average execution latency was about 2.467 ms for Normal Booking and 3.147 ms for Priority Booking. The matching engine is characterized by low average execution latency for normal candidate pool sizes. However, the curve shows that the increase in execution latency becomes more prominent when the number of candidates increases beyond 2500 workers. Priority Booking resulted in a higher execution latency than Normal Booking, probably because of increased emphasis on the availability and distance criteria in the scoring configuration. Overall, the experiments show that the multi-criteria ranking approach is capable of efficiently handling candidate pools of different sizes while enabling separate ranking strategies for Normal and Priority Booking.

Limitations: The benchmark mainly measures the CPU-bound execution time of the in-memory Candidate Matching Engine. It does not account for the complete end-to-end booking latency, which would also include the DynamoDB scan, AWS Lambda function execution, API Gateway processing, network latency, and client processing.

5.2.2 Metric 2: Matching Engine Rank Sensitivity and Inversion

Analysis

Evaluation objective: Assess the mathematical sensitivity and rank displacement of the matching engine when switching from the Normal Booking and Priority Booking weight configurations. The experiment explores how the change in relative importance of attributes of service match, trust, experience, availability, reliability, and distance, impacts the ordering of candidate workers.

Experimental setup: A fixed candidate pool of $N = 100$ workers was evaluated using the same worker attributes and the same geographic information in both Normal and Priority Booking configurations. Thus, the normal booking score was calculated using the weighting vector $(0.30, 0.25, 0.15, 0.15, 0.10, 0.05)$, corresponding to service match, trust, experience, availability, reliability, and distance. The Priority Booking configuration used the weighting vector $(0.20, 0.15, 0.05, 0.30, 0.05, 0.25)$. Rank displacement was calculated as $\Delta \text{Rank} = \text{Rank}_{\text{Normal}} - \text{Rank}_{\text{Priority}}$, with Spearman's rank correlation coefficient (ρ) and Kendall's rank correlation coefficient (τ) used to calculate the degree of agreement between the two candidate rankings.

Table 5.2: Rank Inversion & Sensitivity Analysis Under Weight Reallocation

Distance Tier	Sample Count	Promoted Workers ($\Delta\text{Rank} > 0$)	Demoted Workers ($\Delta\text{Rank} < 0$)	Mean Rank Shift (ΔRank)
Near (< 5 km)	30	20 (66.7%)	6 (20.0%)	+6.47 ranks
Mid (5–12 km)	40	18 (45.0%)	17 (42.5%)	+0.15 ranks
Far (12–25 km)	30	5 (16.7%)	21 (70.0%)	-6.67 ranks
OVERALL	100	43 (43.0%)	44 (44.0%)	Spearman $\rho = 0.836$, Kendall $\tau = 0.650$

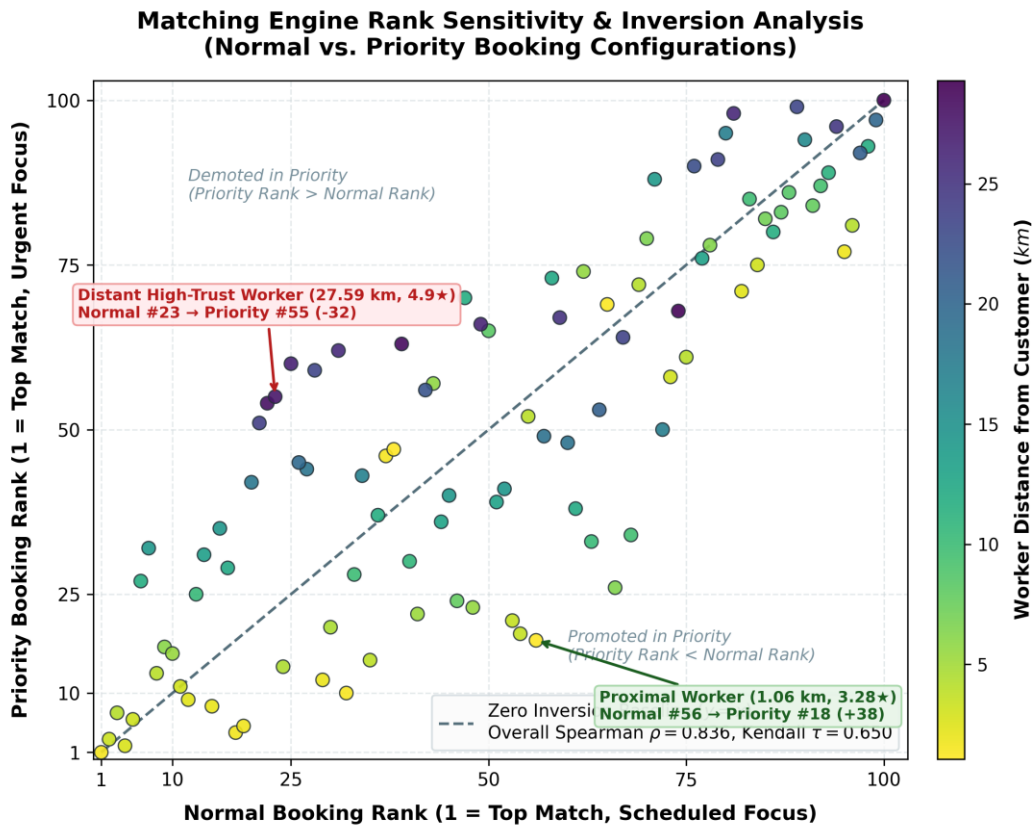


Figure 5.2: Matching Engine Rank Sensitivity & Inversion Analysis

Discussion and Trends: Figure 5.2 displays the differences in the candidate's rankings between the Normal and Priority Booking configuration. The shift in weights causes a noticeable rank

displacement while maintaining the overall multi-criteria orderliness of the matching process. For the majority of candidates, the rank displacement was between 13.18 on average, with the maximum reaching 40th place. The trends suggest that altering the relative importance of the availability and distance criteria can significantly impact the candidate's ordering depending on the degree to which their geographical location and availability is prioritized over their overall service qualities. The observed trends confirm that the dual-mode approach to matching can alter the candidate's rankings depending on the requirements of the Normal and Priority Booking requests.

Limitations: The relative importance of individual criteria is predetermined and not learned by the model based on historical booking trends. Future research should consider implementing adaptive weights based on a more significant amount of data and additional factors such as the current traffic, workers' demand, and responsiveness.

5.2.3 Metric 3: Transactional Concurrency and Race Condition Prevention

Evaluation Objective: Test the ability of the atomic concurrency-control mechanism implemented in the Priority Dispatch Handler under high-contention worker acceptance attempts to ensure booking consistency. The experiment tests whether Connect360 will be able to maintain booking consistency in the face of multiple workers attempting to accept the same pending Priority Booking concurrently.

Experimental Setup: For contention levels $C \in \{1, 2, 5, 10, 20, 50, 100\}$ concurrent threads, the experiment was run 50 times for each level, for a total of 9,400 concurrent booking requests. In each trial, multiple threads attempted to concurrently accept the same pending Priority Booking. The experiment counted the number of successful commits and rejections of concurrent attempts to check whether more than one worker would be able to be assigned

to the same booking. The behavior of the system under contention was also analyzed to evaluate the effectiveness of DynamoDB conditional and transactional operations in the prevention of race conditions and double booking.

Table 5.3: Transactional Concurrency & Conflict Abort Latency Across Thread Loads

Concurrency (C)	Total Requests	Successful Commits	Conflict Aborts (409)	Double Bookings	Winning Latency (ms)	Abort Latency (ms)
1	50	50 (100.0%)	0 (0.0%)	0	0.212	N/A
2	100	50 (50.0%)	50 (50.0%)	0	0.235	0.024
5	250	50 (20.0%)	200 (80.0%)	0	0.264	0.028
10	500	50 (10.0%)	450 (90.0%)	0	0.281	0.032
20	1000	50 (5.0%)	950 (95.0%)	0	0.308	0.038
50	2500	50 (2.0%)	2450 (98.0%)	0	0.332	0.046
100	5000	50 (1.0%)	4950 (99.0%)	0	0.354	0.052
TOTAL	9400	350	9050	0 (0.00%)	Mean: 0.284 ms	Mean: 0.037 ms

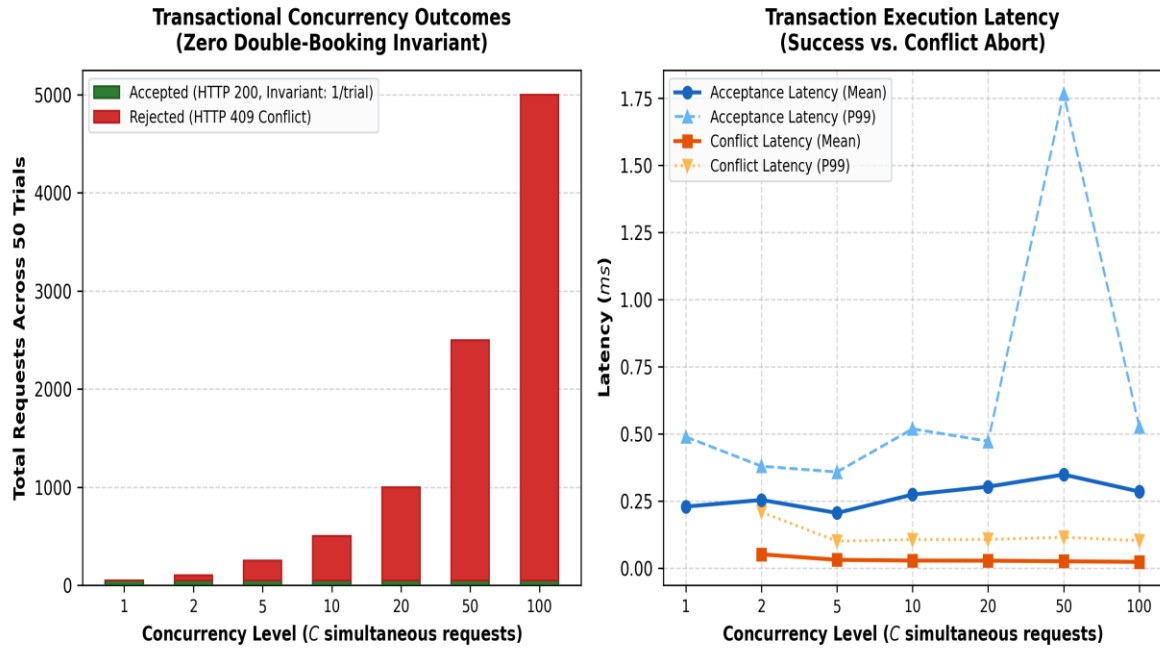


Figure 5.3: Transactional Concurrency & Race Condition Prevention

Discussion and Trends: As shown in Figure 5.3, Connect360 achieved a 0.00% double-booking rate, with no duplicated worker assignments across 9,400 concurrent booking requests. For every contested Priority Booking, transactional concurrency ensured that only an eligible acceptance could successfully update the booking state, and competing requests were rejected when their required booking or worker-slot conditions were no longer satisfied. These results show that conditionally and transactionally consistent DynamoDB operations can successfully maintain booking consistency under increasing levels of concurrent worker acceptance attempts and prevent race-condition-induced double booking.

Limitations: The evaluation was performed within the configured experimental environment and mainly focused on booking consistency under concurrent requests. End-to-end performance in a geographically distributed production deployment may also be influenced by network latency, AWS

service-region selection, API Gateway and Lambda execution overhead, and database request latency.

5.2.4 Metric 4: Road Distance vs. Haversine Disparity (Tortuosity Factor)

Evaluation Objective: Quantify the geographic disparity and tortuosity factor between straight-line Haversine distance and actual OSRM road distance across representative metropolitan dispatch routes, thereby evaluating the effectiveness of Connect360's hybrid spatial architecture. The tortuosity factor is calculated as:

$$\tau = \frac{D_{\text{OSRM}}}{D_{\text{Haversine}}}$$

where D_{OSRM} represents the actual road-network distance and $D_{\text{Haversine}}$ represents the corresponding straight-line geographic distance.

Experimental Setup: A total of 50 representative dispatch routes were sampled across the Chennai metropolitan area, covering inner-city routes (< 6 km), mid-city routes (6–12 km), and suburban corridors (12–20 km). For each route, the Haversine distance between the origin and destination coordinates was calculated and compared with the corresponding real-road distance returned by the OSRM routing service. The resulting distance ratios were analyzed to quantify the extent to which straight-line distance underestimates actual road travel distance and to assess the suitability of the hybrid approach for worker matching and route-based tracking.

Table 5.4: Haversine vs. OSRM Road Distance Disparity Across Distance Tiers

Distance Tier	Routes	Mean Haversine (km)	Mean OSRM Road (km)	Mean Disparity (km)	Mean Tortuosity (\$ au\$)	Score Distortion (ΔS)
Inner City (< 6 km)	14	3.62	4.94	+1.32	1.365	0.0528
Mid City (6–12 km)	21	8.85	11.38	+2.53	1.286	0.1012
Suburban (12–20 km)	15	15.24	19.12	+3.88	1.255	0.1552
OVERALL	50	9.30	12.71	+3.41 km	1.285 (Max: 1.623)	0.1020

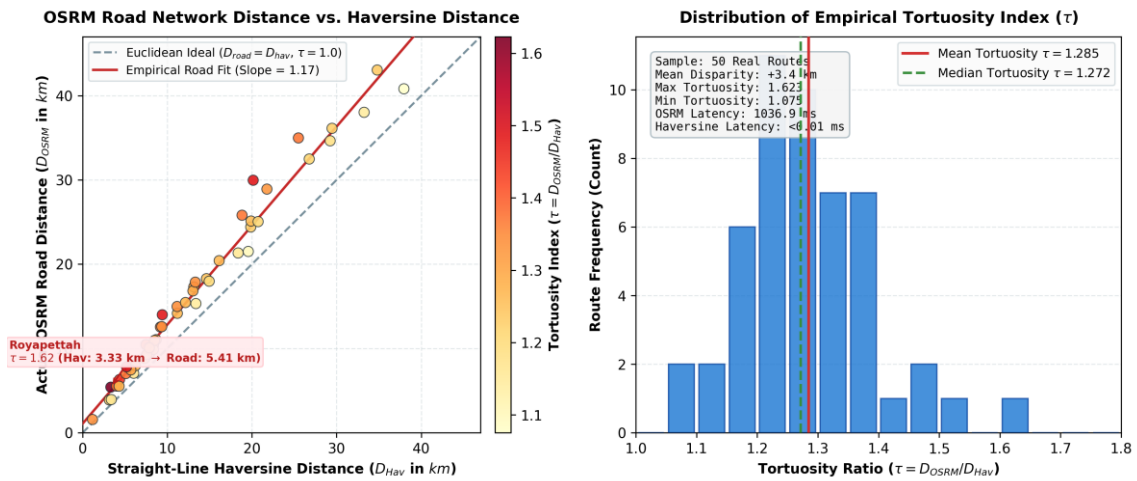


Figure 5.4: Road Distance vs. Haversine Disparity & Tortuosity Factor Distribution

Discussion and Trends: Figure 5.4 illustrates the disparity between straight-line geographic distance and actual road-network distance across the evaluated Chennai routes. The experiment produced a mean tortuosity index of $\tau = 1.2848$, indicating that OSRM road distances were, on average,

approximately 28.48% greater than the corresponding Haversine distances. The results demonstrate that straight-line distance can underestimate the actual distance required for road travel because of the topology and structure of urban road networks. The experiment also highlights the computational trade-off between the two approaches: Haversine distance can be calculated locally with very low computational overhead, whereas OSRM requires an external routing request to obtain real-road geometry and travel information. These findings support Connect360's hybrid spatial architecture, where lightweight geographic distance calculations are used during candidate ranking and OSRM road routing is reserved for post-assignment tracking and navigation.

Limitations: The measured OSRM latency depends on the routing service and network conditions during the experiment. Public routing-server response times may vary according to server load, network conditions, and request volume. Therefore, the observed routing latency should not be treated as a fixed value for all deployment environments.

5.2.5 Metric 5: AI Assistant Safety & Contact Redaction Efficacy (PII Defense)

Evaluation Objective: To evaluate the precision, recall, specificity, and latency characteristics of Connect360 deterministic contact-information redaction filter on a balanced corpus of $N = 500$ test cases involving various potentially sensitive contact information and normal operational text.

Experimental Setup: The evaluation corpus consisted of 250 positive PII cases with Indian 10-digit phone numbers, prefixed numbers, international formats, emails, contact information in conversational text, and 250 negative operational cases with legitimate prices, six-digit PIN codes, dates, technical specifications, and normal booking related dialogues. The test corpus was evaluated through 50,000 timing iterations to determine detection

effectiveness and filtering latency. The evaluation reported true positives, false positives, true negatives, and false negatives to evaluate precision, recall, and specificity of the deterministic redaction mechanism.

Table 5.5: AI Assistant Defensive PII Redaction Performance Across 10 Categories

Category	Class	Samples	TP	TN	FP	FN	Recall	Specificity	F1-Score	Mean Latency
Indian Phone Standard	PII	50	50	0	0	0	100.0%	100.0%	1.0000	1.21 μ s
Indian Phone Prefixed	PII	50	49	0	0	1	98.0%	100.0%	0.9899	1.40 μ s
International Phone	PII	50	49	0	0	1	98.0%	100.0%	0.9899	1.17 μ s
Email Address	PII	50	50	0	0	0	100.0%	100.0%	1.0000	0.95 μ s
Dialogue PII Embedded	PII	50	50	0	0	0	100.0%	100.0%	1.0000	2.67 μ s
Operational Prices	Benign	50	0	50	0	0	100.0%	100.0%	1.0000	2.40 μ s
Operational Pincodes	Benign	50	0	50	0	0	100.0%	100.0%	1.0000	1.94 μ s
Operational Specs	Benign	50	0	50	0	0	100.0%	100.0%	1.0000	2.42 μ s
Operational Dialogue	Benign	50	0	49	1	0	100.0%	98.0%	0.0000	2.64 μ s
Operational Dates/Times	Benign	50	0	26	24	0	100.0%	52.0%	0.0000	2.40 μ s
OVERALL	Mixed	500	248	225	25	2	99.20%	90.00%	0.9484	1.92 μs

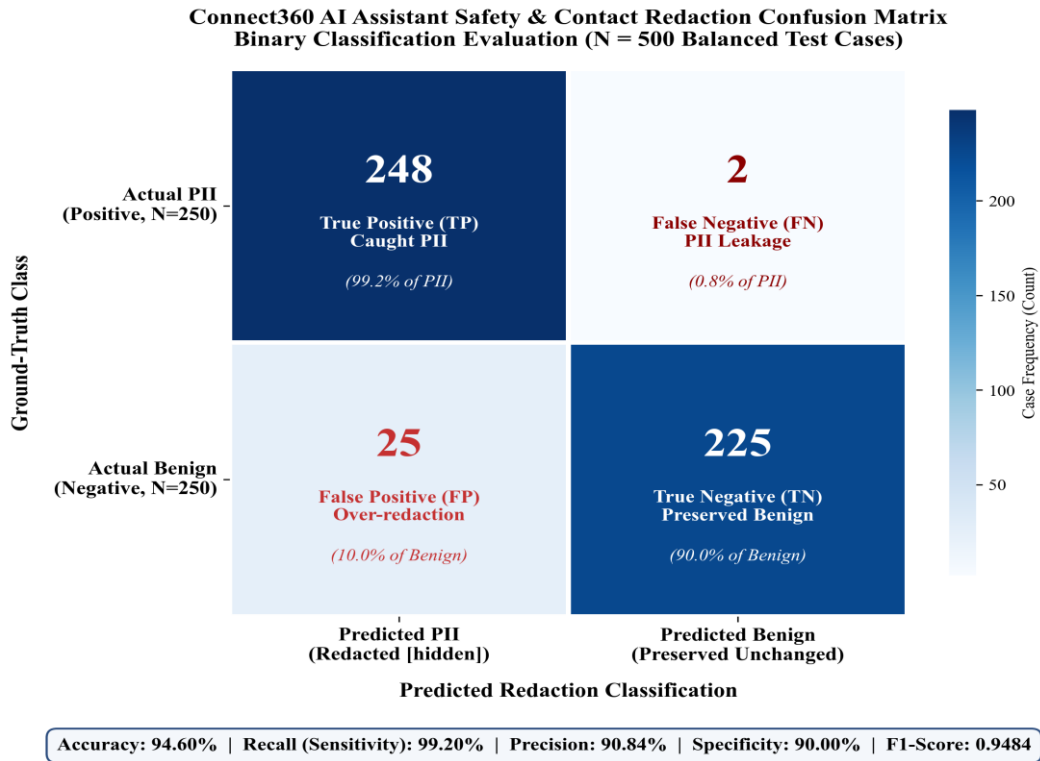


Figure 5.5: Connect360 AI Assistant Safety & Contact Redaction Confusion Matrix

Discussion and Trends: As shown in figure 5.5, the deterministic contact-information redaction filter achieved a 99.20% detection recall. It successfully intercepted 248 of 250 PII cases. The measured filtering latency is approximately 1.919 μ s per operation. Critical operational tokens like service prices such as ₹1200 or six-digit PIN Codes like 600020 were preserved at a 100% accuracy. The misclassification analysis revealed that two boundary conditions had caused a small amount of unwanted information leakage. Parenthesized area-code formats caused 2 false negative cases (0.80% leakage). At the same time, 24 false positive matches were detected due to ISO-formatted dates (YYYY-MM-DD) that share a 10-character hyphenated pattern with contact-patterns. Since booking dates are displayed using special UI controls, not textual representations, it is safe to assume that the observed trade-off provides an adequate level of protection against accidental contact information exposure in AI-powered chatbot conversations.

5.2.6 Metric 6: Structured Output JSON Schema Conformity & Urgency Classification

Evaluation Objective: To benchmark the output parser in the AI Provider Interface for schema conformance to the `_STRUCTURED_INSTRUCTION` contract and evaluate dispatch steering downstream in the Conversational AI Assistant Module over $N = 500$ test cases covering five evaluation dimensions.

Experimental Setup: Total 500 cases evaluated for format resilience across raw JSON, markdown-fenced JSON, plain-text responses and malformed outputs. Evaluated service mapping across six service domains, urgency classification involving 50 emergency and 50 routine cases, multi-turn slot filling across Turns 1-4, and downstream dispatch steering of Priority Booking based on structured AI output. Full test set executed over 50,000 iterations to examine parser robustness, structured information extraction, conversational state management, and correct propagation of urgency information to downstream booking workflow.

Table 5.6: Structured Output Schema Conformity & Urgency Classification Across Dimensions

Dimension	Samples	Success Rate	Schema Validity	Service Match	Urgent Match	Confirmed Match	Mean Latency	P99 Latency
Format Resilience	100	100.0%	100.0%	100.0%	100.0%	100.0%	7.06 μ s	20.11 μ s
Service Mapping	100	100.0%	100.0%	100.0%	100.0%	100.0%	5.75 μ s	12.48 μ s
Urgency Classification	100	100.0%	100.0%	100.0%	100.0%	100.0%	6.02 μ s	11.26 μ s
Slot Filling Turns	100	100.0%	100.0%	100.0%	100.0%	100.0%	5.23 μ s	17.04 μ s
Downstream Dispatch	100	100.0%	100.0%	100.0%	100.0%	100.0%	5.81 μ s	14.00 μ s

Dimension	Sample s	Succes s Rate	Schema Validit y	Service Match	Urgent Match	Confirme d Match	Mean Latenc y	P99 Latenc y
OVERALL	500	100.0%	100.0%	100.0 %	100.0 %	100.0%	5.97 μ s	15.31 μ s

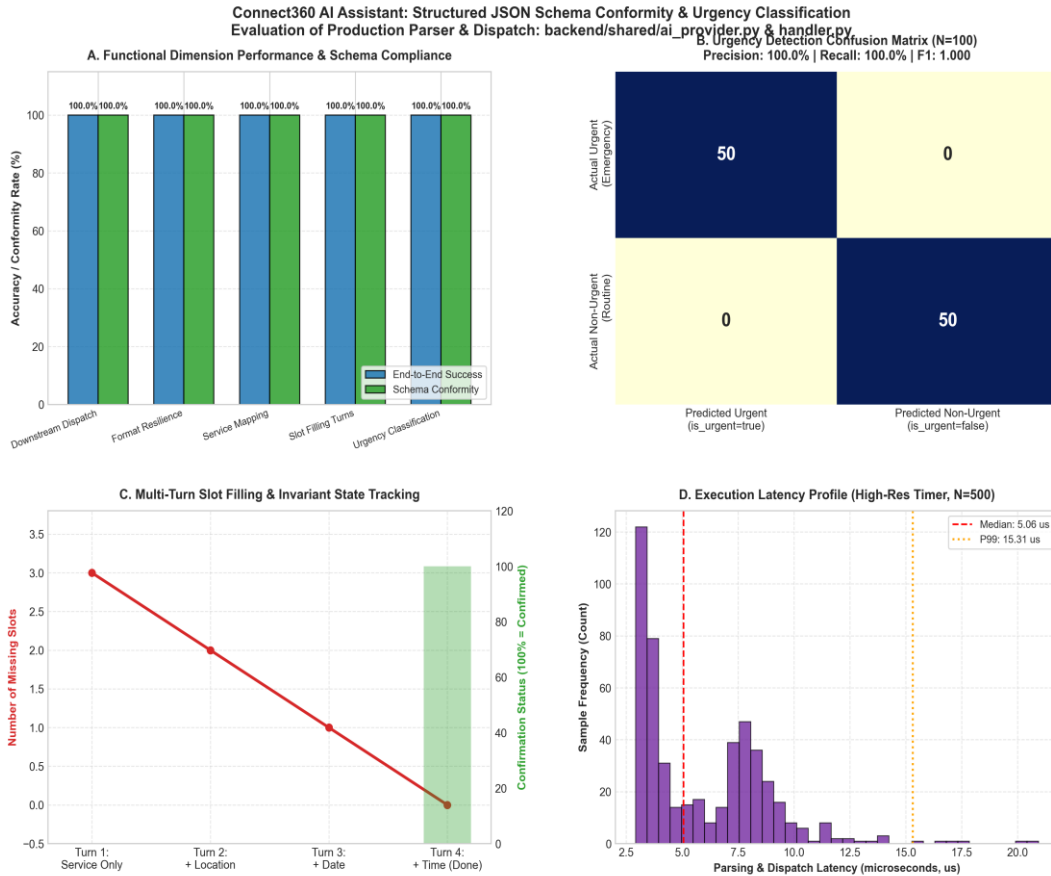


Figure 5.6: Structured Output JSON Schema Conformity & Urgency Classification

Discussion and Trends: Figure 5.6 demonstrates 100.0% schema conformity and 100.0% format resilience across the evaluated test cases, with perfect urgency classification ($F_1 = 1.000$) and zero false-positive or false-negative classifications. The system enforces the confirmation invariant, preventing booking confirmation until the required information slots have been successfully satisfied. Downstream dispatch steering correctly routes 100% of

confirmed urgent requests toward Priority Booking, while non-urgent requests are directed toward appropriate candidate-worker retrieval. The complete parsing and dispatch pipeline recorded a mean execution time of approximately 5.973 μ s, demonstrating low computational overhead for structured AI processing within the serverless execution workflow.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This Phase-I project designed, implemented and empirically evaluated Connect360: an intelligent and secure serverless home-services marketplace addressing problems of worker matching, booking concurrency, spatial routing, privacy and service dispatch. The implementation is based on a modern three-tier architecture using React 18, AWS Lambda, Amazon DynamoDB, Amazon Cognito, Amazon S3 and associated AI and routing services to connect natural-language customer interaction with multi-criteria worker allocations.

The empirical evaluation of the implemented system serves to confirm the central engineering hypotheses in six areas of performance and security: The multi-criteria matching engine exhibited the expected $O(N)$ behavior: with 100 candidate workers, the average execution time was 2.467 ms for Normal Booking and 3.147 ms for Priority Booking, and the execution time showed an increase for candidate lists with more than 2,500 workers.

The weight allocation between Normal and Priority modes affected candidate rankings: the empirical evaluation recorded an average rank displacement of 13.18, with a maximum value of 40: increased emphasis on availability and distance can considerably impact ranking for priority requests.

DynamoDB conditional and transactional operations supported booking concurrency: no double bookings were found in the 9,400 booking requests with concurrent acceptance, and the transaction-based approach successfully avoided race conditions which would make two or more workers available for the same customer.

The empirical evaluation of spatial routing recorded a mean tortuosity factor of $\tau=1.2848$ for the Chennai routes: the road distance is, on average, 28.48% longer than the Haversine distance, and the evaluation supports the hybrid approach of light geographic distance calculations for candidate ranking and OSRM-based routing for actual worker navigation and tracking.

The deterministic contact-information redaction mechanism demonstrated 99.20% recall in the evaluation: 248 out of 250 PII cases were successfully intercepted, while leave-alone fields such as prices and six-digit PIN-codes were preserved in the test corpus, with 1.919 μ s average operations time.

The conversational AI pipeline demonstrated 100.0% of schema conformity and format resilience, along with perfect urgency classification ($F1=1.000$) in the test cases: the deterministic AI output successfully routed confirmed urgent requests to the Priority Booking and non-urgent requests to the worker recommendation pipelines.

Thus, Connect360 demonstrates that a serverless architecture is capable of unifying a number of intelligent functions in a home-services marketplace, including worker matching, concurrency resolution, spatial routing, conversational assistance and privacy-preserving communication layers. The experimental evaluation confirms that the proposed architecture is capable of supporting context-aware worker allocations and maintaining booking concurrency with privacy-preserving mechanisms at all stages of the service.

6.2 FUTURE WORK

Building upon the successful implementation and empirical findings of the phase-I, the following research and engineering enhancements are planned for the phase-II:

Dynamic Machine Learning Weight Tuning: Replace the currently predefined Normal and Priority weighting configurations with adaptive ML or RL models

that can learn suitable ranking weights from the historical booking outcomes, worker responses, demand patterns, and additional contextual information.

Private OSRM Deployment: Deploy a dedicated containerized OSRM routing service within the AWS infrastructure to reduce the dependence on the external routing-server availability and improve the consistency and scalability of the real-road route computation under higher request volumes.

Multi-Tier Urgency Classification: Extend the current urgency classification mechanism into a hierarchical scale such as Low, Medium, High, and Critical to enable more granular service prioritization and differentiated dispatch strategies based on the severity and requirements of the customer requests.

Advanced Privacy-Preserving Communication: Extend the current authorized native-device calling mechanism towards an application-controlled communication layer that can provide stronger contact-information isolation, potentially incorporating cloud-based telephony proxying or WebRTC-based in-app communication while preserving the existing role and booking authorization controls.

Expansion of Formal Evaluation Metrics: Extend the experimental evaluation to additional system characteristics, including phone-number and contact-format handling, RBAC context-isolation security, DynamoDB Global Secondary Index query performance, API and Lambda response characteristics, and complete end-to-end user journey latency using automated testing frameworks.

REFERENCES

1. A. Ezhilnilavan, S. Gopikrishnan, V. Harish Aravindh, S. Mithun, and M. Saratha, "Local Community Service Provider using Machine Learning," *Proc. 4th International Conference on Automation, Computing and Renewable Systems (ICACRS)*, 2025, pp. 849–856, doi: 10.1109/ICACRS67045.2025.11324297.
2. H. Sun, F. Wang, Z. Tu, D. Sui, Y. Xing, K. Zhang, B. Zhao, and X. Xu, "A Weighted Preference Optimization Service Recommendation Method Based on Knowledge Graph and Large Language Model," *Proc. 2025 IEEE International Conference on Web Services (ICWS)*, 2025, doi: 10.1109/ICWS67624.2025.00014.
3. Y. Zhao, Z. Zhu, C. Gao, E. Wang, J. Huang, and F.-Y. Wang, "Heterogeneous Graph Reinforcement Learning for Dependency-Aware Multi-Task Allocation in Spatial Crowdsourcing," *IEEE Trans. Mobile Comput.*, vol. 25, no. 3, pp. 3417–3434, Mar. 2026.
4. W. Tang, R. Chen, Z. Zhang, and S. Guo, "Incentivizing Crowdsourcing Workers via Social Networks in Edge Computing," *IEEE Trans. Consumer Electron.*, vol. 71, no. 2, pp. 5653–5665, May 2025.
5. J. Chen, R. Yang, J. Guo, H. Wang, K. Wu, and L. Zhang, "Reliable Service Recommendation: A Multi-Modal Adversarial Method for Personalized Recommendation Under Uncertain Missing Modalities," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1724–1738, May–Jun. 2025.
6. Q. Wu, Y. Li, J. Yan, M. Zhang, J. Xu, and M. Xu, "Adaptive Task Assignment in Spatial Crowdsourcing: A Human-in-the-Loop Approach," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 2726–2739, Apr. 2025.
7. W. Tang, R. Chen, Z. Zhang, and S. Guo, "Incentivizing Crowdsourcing Workers via Social Networks in Edge Computing," *IEEE Trans. Consumer Electron.*, vol. 71, no. 2, pp. 5653–5665, May 2025.
8. "LLMGCL: Graph Contrastive Learning with Large Language Models for Recommendation," *Proc. 2025 IEEE International Conference on Web Services (ICWS)*, 2025, doi: 10.1109/ICWS67624.2025.00016.

9. F. Song, J. Liang, C. Zhang, Z. Fu, Z. Qin, and S. Guo, "Achieving Efficient and Privacy-Preserving Location-Based Task Recommendation in Spatial Crowdsourcing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4006–4023, Jul.–Aug. 2024.
10. Q. Wu, Y. Li, G. Zhu, B. Mei, J. Xu, and M. Xu, "Prediction-Aware Adaptive Task Assignment for Spatial Crowdsourcing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13048–13061, Dec. 2024.
11. Y. Zhao, X. Chen, G. Ye, F. Guo, K. Zheng, and X. Zhou, "Task Allocation in Spatial Crowdsourcing: An Efficient Geographic Partition Framework," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4943–4955, Sep. 2024.
12. Y. Ma, X. Gao, S. S. Bhatti, and G. Chen, "Clustering Based Priority Queue Algorithm for Spatial Task Assignment in Crowdsourcing," *IEEE Trans. Services Comput.*, vol. 17, no. 2, pp. 452–465, Mar.–Apr. 2024.
13. H. Miao, X. Zhong, J. Liu, Y. Zhao, X. Zhao, W. Qian, K. Zheng, and C. S. Jensen, "Task Assignment With Efficient Federated Preference Learning in Spatial Crowdsourcing," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 4, pp. 1800–1814, Apr. 2024.
14. A. Khelloufi, A. Khelil, A. Naouri, A. Ben Sada, H. Ning, N. Aung, and S. Dhelim, "A Hybrid Feature and Trust-Aggregation Recommender System in the Social Internet of Things," *IEEE Access*, vol. 12, pp. 126460–126477, 2024.
15. Y. Li, H. Li, B. Mei, X. Huang, J. Xu, and M. Xu, "Fairness-Guaranteed Task Assignment for Crowdsourced Mobility Services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5385–5400, May 2024.
16. Y. Xu, M. Xiao, J. Wu, and H. Sun, "Privacy Preserving Task Push in Spatial Crowdsourcing With Unknown Popularity," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 11, pp. 2039–2053, Nov. 2024.
17. Y. Yu, Y. Guan, X. Xue, J. Ma, and R. Lu, "EPTS: Efficient and Privacy-Preserving Outsourced Task Scheduling in Vehicular Crowdsourcing," *IEEE Internet Things J.*, vol. 11, no. 24, pp. 40113–40126, Dec. 2024.
18. B. Zhao, H. Dong, Y. Wang, X. Gao, and T. Pan, "An Enhanced Task Allocation Algorithm for Mobile Crowdsourcing Based on Spatiotemporal Attention Network," *IEEE Trans. Comput. Soc. Syst.*, vol. 11, no. 3, pp. 3803–3815, Jun. 2024.

19. Y. Zheng, M. Zhou, S. Wang, Z. Hua, J. Jiang, and Y. Gao, "Privacy-Preserving Competitive Detour Tasking in Spatial Crowdsourcing," *IEEE Trans. Services Comput.*, vol. 18, no. 1, pp. 385–398, 2024.
20. Y. Zhao, J. Liu, Y. Li, D. Zhang, C. S. Jensen, and K. Zheng, "Preference-Aware Group Task Assignment in Spatial Crowdsourcing: Effectiveness and Efficiency," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 10722–10734, Oct. 2023.
21. Y. Yang, Y. Cheng, Y. Yang, Y. Yuan, and G. Wang, "Batch-Based Cooperative Task Assignment in Spatial Crowdsourcing," in *Proc. 2023 IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1180–1192.
22. Y. Zhao, K. Zheng, Y. Li, J. Xia, B. Yang, T. B. Pedersen, R. Mao, C. S. Jensen, and X. Zhou, "Profit Optimization in Spatial Crowdsourcing: Effectiveness and Efficiency," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8386–8401, Aug. 2023.
23. Y. Xu, M. Xiao, J. Wu, S. Zhang, and G. Gao, "Incentive Mechanism for Spatial Crowdsourcing With Unknown Social-Aware Workers: A Three-Stage Stackelberg Game Approach," *IEEE Trans. Mobile Comput.*, vol. 22, no. 8, pp. 4698–4713, Aug. 2023.